

Learn Python

A 14-Day / 28-Hour Job-Ready Curriculum

Two hours a day for two weeks. Full depth on the 'why', hands-on exercises every day, a Week 1 project, and a Week 2 capstone. Companion files live in `D:/Git/learning-code/python/`

Week	Focus	Days
Week 1	Core language foundations	Days 1-7
Week 2	Job-ready practices: OOP, testing, real data, packaging	Days 8-14

How To Use This Track

Each day lives in its own folder: `dayNN-topic/`. Every folder has a `lesson.md` (read first, ~35 min), an `exercises.py` with TODO blanks you fill in and run yourself (~70 min), and a `solutions.py` reference (~15 min to compare against, after you've genuinely attempted it). Struggling with a problem for 10-15 minutes before checking the answer is where the actual learning happens -- don't skip that step.

Setup: you need Python 3.10+ (verify with `python --version`). No extra packages until Day 11 (requests) and Day 12 (pytest) -- each lesson tells you exactly what to install when you get there.

The Daily Rhythm (~2 hours)

- Read `lesson.md`, running every example yourself in a Python REPL as you go (35 min)
- Complete the exercises in `exercises.py` -- run it and iterate on PASS/FAIL output (70 min)
- Compare against `solutions.py`; for anything wrong, write one sentence on WHY (15 min)

Week 1 -- Foundations

Day 1: Variables, Types, and the Object Model

Variables as names (not boxes); mutability vs immutability; `id()` vs `==`; how Python passes arguments; the core built-in types; dynamic vs duck typing; truthiness.

Day 2: Control Flow: Conditionals, Loops, Comprehensions

`if/elif/else` and comparison chaining; `for` as 'iterate,' not 'count'; `enumerate/zip`; `while` vs `for`; `break/continue/for-else`; list/dict/set comprehensions and when to avoid them; the `match` statement.

Day 3: Data Structures: Lists, Tuples, Dicts, Sets

When to reach for each container; hash tables and why dict/set lookup is $O(1)$; why dict keys must be hashable; slicing in depth; the list/dict/set methods used daily.

Day 4: Functions: Args, Scope, Closures, First-Class Functions

`*args/**kwargs` and parameter ordering; the mutable default argument trap; the LEGB scope rule and 'global'/'nonlocal'; closures and the late-binding loop gotcha; functions as first-class objects; lambdas.

Day 5: Strings, Formatting, Files, and Context Managers

Core string methods; f-strings and format specs; text vs binary file modes; why 'with' exists (`__enter__`/`__exit__` and guaranteed cleanup); encoding pitfalls.

Day 6: Errors, Exceptions, Modules, and Packages

EAFP vs LBYL; `try/except/else/finally`; custom exception classes and exception chaining; modules, packages, and imports; the `if __name__ == '__main__':` pattern.

Day 7: Mini Project: CLI Task Tracker (Week 1 Review)

A from-scratch command-line to-do app with JSON persistence, combining every Week 1 topic. Less hand-holding by design -- this is where independent problem-solving starts.

Week 2 -- Job-Ready Python

Day 8: OOP Fundamentals: Classes, self, Instances

Classes as factories for objects; why self is explicit, not magic; `__init__`; class attributes vs instance attributes (and the shared-mutable-default bug); when a class beats a dict or a function.

Day 9: OOP Advanced: Inheritance, Dunder Methods, Composition

Inheritance and `super()`; polymorphism and `isinstance` vs `type()`; dunder methods (`__repr__`, `__str__`, `__eq__`, `__add__`, and more) that hook your classes into built-in syntax; composition vs inheritance and when to prefer each.

Day 10: Iterators, Generators, and Decorators

The iterator protocol behind every for loop; generator functions with `yield` and why they're memory-efficient; decorators as closures wrapping functions; `functools.wraps`; decorators that take their own arguments.

Day 11: Working with Real Data: JSON, CSV, HTTP APIs, Virtual Environments

`json.dump(s)/load(s)` in depth; the `csv` module (and why not to parse it by hand); making HTTP requests with the `requests` library, timeouts, and error handling; what a virtual environment is and why every real project uses one.

Day 12: Testing and Debugging with pytest, Type Hints

Why automated tests matter beyond 'does it work once'; `pytest` assertions, `pytest.raises`, fixtures, and `parametrize`; what type hints do (and don't) give you; using a real debugger (`breakpoint()`) instead of scattered `print()` calls.

Day 13: Practical Python: Project Structure, Packaging, Git Workflow

The conventional `src`-layout; `pyproject.toml` and dependency management; `.gitignore` and why `__pycache__/.venv` are never committed; the real git branch + commit + PR workflow applied to a Python project; linting/formatting with `ruff` and `black`.

Day 14: Capstone Project: Expense Tracker

A complete CLI application built mostly unassisted: OOP models, a generator-based filter method, a hand-built logging decorator, JSON persistence, CSV export, custom exceptions, type hints throughout, and a real `pytest` suite -- proof of end-to-end fluency.

What You Should Be Able to Do After Day 14

- Read and reason about someone else's Python code, including idioms like comprehensions, context managers, decorators, and generators.
- Design a small program with classes when appropriate, and justify composition vs inheritance choices.
- Handle errors deliberately (custom exceptions, EAFP) instead of letting a program crash on bad input.
- Read/write JSON and CSV, and call a real HTTP API with proper timeout and error handling.
- Write and run a pytest suite covering happy paths and edge cases for your own code.
- Set up a project with a virtual environment, pyproject.toml, and a sane git workflow -- the same baseline expected on day one of a Python job.

Full lessons, runnable exercises, and reference solutions for every day are in `D:/Git/learning-code/python/`. Start with `day01-basics/lesson.md`.